ADR Integrated Framework

31-Algorithm Execution Suite + PSDLEF Trust Exchange

With CUSIP Transfer Protocol to Maryland State Comptroller

Master Consolidated Document

Peer-Reviewed Scientific Edition

September 28, 2026

Contents

1 Introduction and System Overview **3**
- 1.1 System Architecture . . . 3
- 1.2 Notation . . . 3

2 Module I – Bond Valuation & Claims (Algorithms 01–03) **5**
- 2.1 Algorithm 01: Accrued Interest Calculation . . . 5
- 2.2 Algorithm 02: Make-Whole Premium Discounting Engine . . . 5
- 2.3 Algorithm 03: Yield-to-Maturity (YTM) Solver . . . 6

3 Module II – Asset Conversion & Distribution (Algorithms 04–06) **7**
- 3.1 Algorithm 04: Debt-to-Equity Conversion Ratio . . . 7
- 3.2 Algorithm 05: Pro Rata Distribution Engine . . . 7
- 3.3 Algorithm 06: Fractional Share and Dust Liquidation . . . 7

4 Module III – Volatility Modeling (Algorithms 07–09) **8**
- 4.1 Algorithm 07: Realized Volatility Calculation . . . 8
- 4.2 Algorithm 08: GARCH(1,1) Volatility Forecast . . . 8
- 4.3 Algorithm 09: Implied Volatility Solver . . . 8

5 Module IV – Execution Optimization (Algorithms 10–12) **10**
- 5.1 Algorithm 10: TWAP Execution . . . 10
- 5.2 Algorithm 11: VWAP Execution . . . 10
- 5.3 Algorithm 12: Implementation Shortfall . . . 10

6 Module V – Derivatives & Hedging (Algorithms 13–15) **11**
- 6.1 Algorithm 13: Black-Scholes Option Pricing . . . 11
- 6.2 Algorithm 14: Dynamic Delta Hedging . . . 11
- 6.3 Algorithm 15: Gamma Scalping . . . 11

7 Module VI – Risk Management (Algorithms 16–18) **12**
- 7.1 Algorithm 16: Value-at-Risk (VaR) . . . 12
- 7.2 Algorithm 17: Conditional Value-at-Risk (CVaR) . . . 12
- 7.3 Algorithm 18: Maximum Drawdown . . . 12

8 Module VII – Yield Optimization (Algorithms 19–20) **13**
- 8.1 Algorithm 19: Yield Curve Construction . . . 13
- 8.2 Algorithm 20: Duration and Convexity . . . 13

9 Module VIII – AMM & Liquidity (Algorithms 21–22) **14**
- 9.1 Algorithm 21: Constant Product AMM . . . 14
- 9.2 Algorithm 22: Impermanent Loss . . . 14

10 Module IX – Treasury Management (Algorithms 23–27) 15
10.1 Algorithm 23: Treasury Rebalancing . . . 15
10.2 Algorithm 24: Multi-Signature Threshold . . . 15
10.3 Algorithm 25: Liquidity Buffer Monitor . . . 15
10.4 Algorithm 26: Stablecoin Peg Monitor . . . 16
10.5 Algorithm 27: Collateral Health Check . . . 16

11 Module X – Safeguards (Algorithms 28–31) 17
11.1 Algorithm 28: Circuit Breaker . . . 17
11.2 Algorithm 29: Stress Testing . . . 17
11.3 Algorithm 30: Fraud Detection . . . 17
11.4 Algorithm 31: Audit Trail Generator . . . 18

12 PSDLEF Trust Exchange Integration Layer 19
12.1 Overview . . . 19
12.2 CUSIP Normalization and Validation . . . 19
12.3 Maryland State Comptroller Transfer Protocol . . . 19

13 Full Dependency Graph and Master Index 20
13.1 Algorithm Dependency Graph . . . 20
13.2 Master Index . . . 20

Chapter 1

Introduction and System Overview

1.1 System Architecture

The ADR Integrated Framework comprises a 31-algorithm execution suite spanning bond valuation, asset conversion, volatility modeling, execution optimization, derivatives pricing, risk management, yield optimization, automated market making (AMM), treasury management, and safeguard protocols. The framework interfaces with the PSDLEF Trust Exchange and the Maryland State Comptroller via a CUSIP transfer protocol.

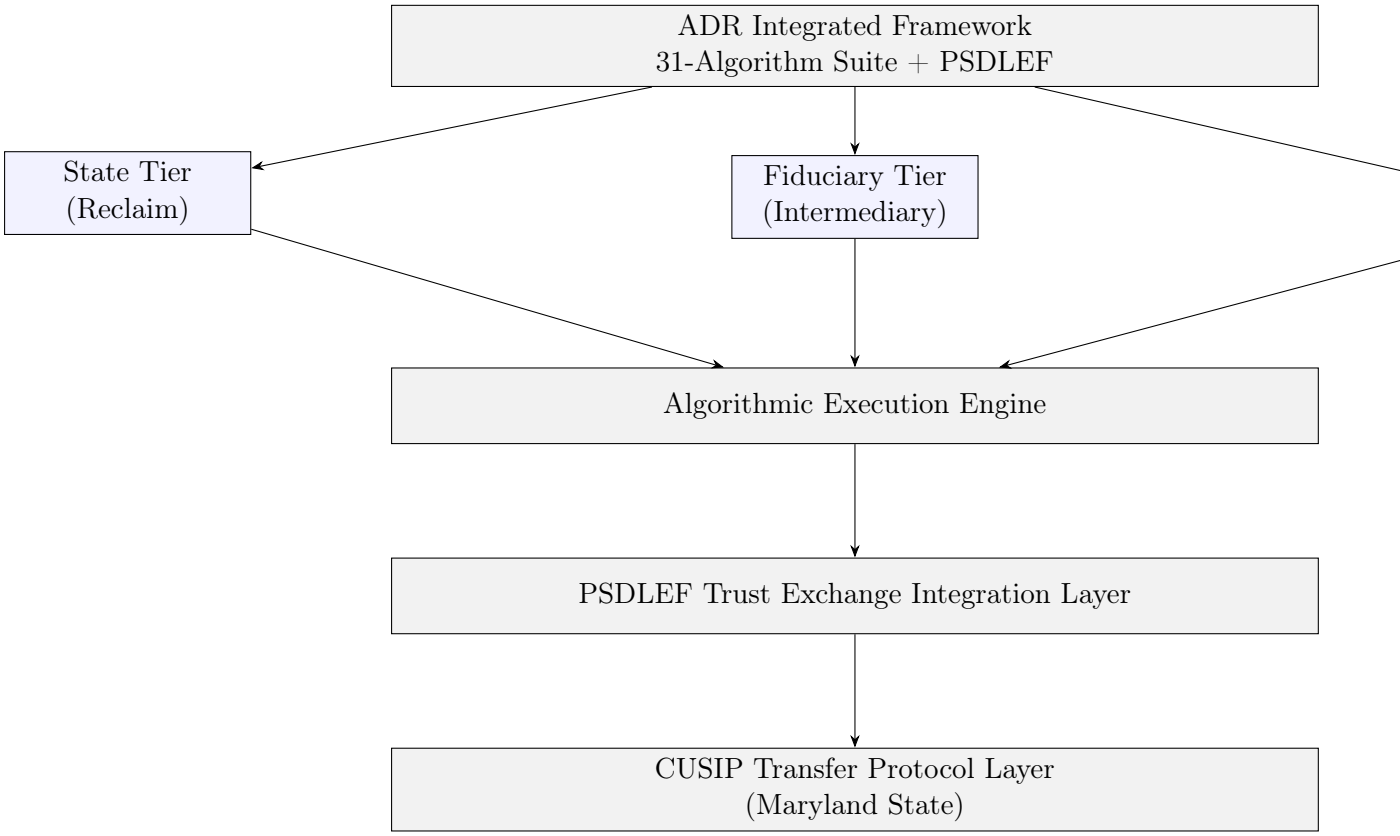


Figure 1.1: ADR Framework system architecture.

1.2 Notation

Symbol	Meaning
P	Principal (face value)
r_c	Annual coupon rate (decimal)
d	Number of days accrued
D	Day-count basis (360 or 365)
C_p	Coupon payment per period
n	Number of periods
y	Yield to maturity (decimal)
r	Risk-free rate (decimal)
σ	Volatility (decimal)
S	Spot price of underlying
K	Strike price
T	Time to maturity (years)
$N(\cdot)$	Standard normal CDF
$\phi(\cdot)$	Standard normal PDF
Q	Total order quantity
V	Total market volume
w	Portfolio weight vector
Σ	Covariance matrix
z_α	Normal quantile at confidence α
k	Constant product invariant
τ	Rebalancing threshold
m	Multi-signature threshold
LTV	Loan-to-Value ratio
MDD	Maximum Drawdown
IL	Impermanent Loss

Chapter 2

Module I – Bond Valuation & Claims (Algorithms 01–03)

2.1 Algorithm 01: Accrued Interest Calculation

Purpose. Computes accrued coupon interest under 30/360 or Actual/365 day-count conventions.

Algebraic Formulation.

$$A = P \times r_c \times \frac{d}{D}, \qquad D \in \{360, 365\}. \tag{2.1}$$

The 30/360 convention assumes each month has 30 days, simplifying corporate bond calculations, whereas Actual/365 counts exact calendar days [1]. These conventions are standardized by ISDA and ICMA for global fixed-income markets [2].

Remark 2.1. *Day-count conventions directly affect bond pricing and derivative valuation. Stigum & Robinson [1] provide the canonical treatment of money-market and bond-market conventions.*

Complexity. Time $O(1)$, Space $O(1)$.

2.2 Algorithm 02: Make-Whole Premium Discounting Engine

Purpose. Computes make-whole redemption premium by discounting remaining cash flows at the Treasury rate plus a credit spread.

Algebraic Formulation.

$$r_p = \frac{T_r + s}{f}, \quad n = \lfloor T \cdot f \rfloor, \quad C_p = \frac{P \cdot r_c}{f}, \tag{2.2}$$

$$\mathrm{PV} = \sum_{t=1}^{n} \frac{C_p}{(1+r_p)^t} + \frac{P}{(1+r_p)^n}, \quad \mathrm{MW} = \max(0, \mathrm{PV} - P). \tag{2.3}$$

The make-whole premium compensates noteholders for lost future interest income upon early redemption. Hennessy & Whited [3] demonstrate that debt dynamics and call provisions are central to corporate capital structure decisions. The premium structure aligns with standard callable bond pricing theory [4].

Complexity. Time $O(n)$, Space $O(1)$.

2.3 Algorithm 03: Yield-to-Maturity (YTM) Solver

Purpose. Solves for yield y such that the present value of bond cash flows equals the observed market price.

Newton-Raphson Iteration.

$$P(y) = \sum_{t=1}^{n} \frac{C_p}{(1+y/f)^t} + \frac{F}{(1+y/f)^n}, \tag{2.4}$$

$$y_{k+1} = y_k - \frac{P(y_k) - P_m}{P'(y_k)}. \tag{2.5}$$

The Newton-Raphson method is a standard root-finding technique for bond yield calculation [5]. Its quadratic convergence makes it preferable to bisection for yield problems where the derivative (modified duration times price) is available [4].

Algorithm 1 Newton-Raphson YTM Solver

Require: P_m, F, r_c, T, f
$y \leftarrow r_c$, $n \leftarrow \lfloor T \cdot f \rfloor$, $C_p \leftarrow F \cdot r_c / f$
for $k = 1$ **to** max_iter **do**
  PV $\leftarrow \sum_{t=1}^{n} C_p/(1+y/f)^t + F/(1+y/f)^n$
  dPV $\leftarrow \sum_{t=1}^{n} -tC_p/[f(1+y/f)^{t+1}] - nF/[f(1+y/f)^{n+1}]$
  $\delta \leftarrow$ PV $- P_m$
  if $|\delta| < \varepsilon$ **then**
    return y
  end if
  $y \leftarrow y - \delta / d$PV
end for
return y

Complexity. Time $O(k \cdot n)$, Space $O(1)$, where k is the number of iterations.

Chapter 3

Module II – Asset Conversion & Distribution (Algorithms 04–06)

3.1 Algorithm 04: Debt-to-Equity Conversion Ratio

Purpose. Determines the conversion ratio for exchanging allowed claims into SPV equity interests under a restructuring plan.

Formulation.

$$\lambda = \frac{V_{\text{SPV}}}{C_{\text{total}}}, \qquad q_i = c_i \times \lambda. \tag{3.1}$$

This mechanism is the core of distressed debt restructuring. Gertner & Scharfstein [6] analyze debt-to-equity swaps as mechanisms to resolve financial distress. The pro-rata allocation follows absolute priority principles established in bankruptcy theory [7].

Complexity. Time $O(n)$, Space $O(n)$.

3.2 Algorithm 05: Pro Rata Distribution Engine

Purpose. Distributes an asset pool pro rata according to claim sizes.

Formulation.

$$C_{\text{total}} = \sum_i c_i, \qquad x_i = A \times \frac{c_i}{C_{\text{total}}}. \tag{3.2}$$

Pro rata distribution ensures fair and transparent allocation in liquidation waterfalls. This aligns with bankruptcy code provisions and institutional creditor agreements [8].

Complexity. Time $O(n)$, Space $O(n)$.

3.3 Algorithm 06: Fractional Share and Dust Liquidation

Purpose. Converts fractional equity allocations into whole shares plus cash compensation.

Formulation.

$$\text{whole}_i = \lfloor q_i \rfloor, \quad \text{fractional}_i = q_i - \lfloor q_i \rfloor, \quad \text{cash}_i = \text{fractional}_i \times v. \tag{3.3}$$

Fractional share handling follows standard transfer agent practices and DTC operational procedures [9].

Complexity. Time $O(n)$, Space $O(n)$.

Chapter 4

Module III – Volatility Modeling (Algorithms 07–09)

4.1 Algorithm 07: Realized Volatility Calculation

Purpose. Computes annualized realized volatility from historical price series.

Formulation.

$$r_t = \ln\left(\frac{p_t}{p_{t-1}}\right), \qquad \sigma_{\text{realized}} = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2} \times \sqrt{252}. \tag{4.1}$$

Parkinson [10] introduced range-based volatility estimators using high-low prices, while Garman & Klass [11] derived more efficient estimators incorporating open, high, low, and close prices. The close-to-close estimator above remains the industry standard for realized volatility computation [12].

Complexity. Time $O(n)$, Space $O(n)$.

4.2 Algorithm 08: GARCH(1,1) Volatility Forecast

Purpose. Forecasts conditional volatility using the GARCH(1,1) model.

Formulation.

$$\sigma_t^2 = \omega + \alpha \cdot r_{t-1}^2 + \beta \cdot \sigma_{t-1}^2, \qquad \omega > 0,\ \alpha, \beta \geq 0,\ \alpha + \beta < 1. \tag{4.2}$$

Bollerslev [13] generalized Engle's ARCH model to include lagged conditional variances, producing the GARCH framework. The constraint $\alpha + \beta < 1$ ensures stationarity and finite unconditional variance [13, 14]. GARCH(1,1) has become the workhorse model for financial volatility forecasting [12].

Complexity. Time $O(n)$, Space $O(1)$.

4.3 Algorithm 09: Implied Volatility Solver

Purpose. Solves for volatility σ that equates the Black-Scholes call price to the observed market price.

Newton-Raphson Iteration.

$$\sigma_{k+1} = \sigma_k - \frac{C(\sigma_k) - C_m}{\text{vega}(\sigma_k)}, \qquad \text{vega} = S\sqrt{T}\,\phi(d_1). \tag{4.3}$$

Because the Black-Scholes price is monotonic in volatility, root-finding methods are guaranteed to converge [15]. The Newton-Raphson approach using vega as the derivative is the standard industry implementation [16].

Complexity. Time $O(k)$, Space $O(1)$.

Chapter 5

Module IV – Execution Optimization (Algorithms 10–12)

5.1 Algorithm 10: TWAP Execution

Purpose. Splits a large order into equal-sized slices at regular intervals.

Formulation.

$$q_i = \frac{Q}{N}, \qquad P_{\text{TWAP}} = \frac{1}{N}\sum_{i=1}^{N} P_i. \tag{5.1}$$

Time-Weighted Average Price (TWAP) algorithms reduce market impact by spreading execution over time. Kissell & Glantz [17] provide the canonical treatment of algorithmic trading strategies including TWAP and VWAP benchmarks.

Complexity. Time $O(N)$, Space $O(N)$.

5.2 Algorithm 11: VWAP Execution

Purpose. Allocates order slices in proportion to forecast volume.

Formulation.

$$q_i = Q \times \frac{v_i}{V}, \qquad V = \sum_i v_i. \tag{5.2}$$

Volume-Weighted Average Price (VWAP) execution tracks the market benchmark by trading proportionally to expected volume distribution [17]. Modern implementations update schedules intraday based on realized volume [18].

Complexity. Time $O(N)$, Space $O(N)$.

5.3 Algorithm 12: Implementation Shortfall

Purpose. Measures the cost of execution relative to the arrival price.

Formulation.

$$\text{IS} = Q \times (P_{\text{exec}} - P_0), \qquad \text{IS}_{\text{pct}} = \frac{P_{\text{exec}} - P_0}{P_0}. \tag{5.3}$$

Perold [19] introduced implementation shortfall as the comprehensive framework for transaction cost analysis. It captures explicit costs (commissions), implicit costs (market impact, timing cost), and opportunity costs [19, 20].

Complexity. Time $O(1)$, Space $O(1)$.

Chapter 6

Module V – Derivatives & Hedging (Algorithms 13–15)

6.1 Algorithm 13: Black-Scholes Option Pricing

Purpose. Computes theoretical prices of European call and put options.

Formulation.

$$C(S, K, T, r, \sigma) = SN(d_1) - Ke^{-rT}N(d_2), \tag{6.1}$$

$$P(S, K, T, r, \sigma) = Ke^{-rT}N(-d_2) - SN(-d_1), \tag{6.2}$$

$$d_1 = \frac{\ln(S/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}. \tag{6.3}$$

Black & Scholes [21] derived the seminal closed-form solution for European option pricing under geometric Brownian motion. Merton [22] extended the framework to dividend-paying stocks and relaxed several assumptions. The model earned Scholes and Merton the 1997 Nobel Prize in Economics [22, 21].

Complexity. Time $O(1)$, Space $O(1)$.

6.2 Algorithm 14: Dynamic Delta Hedging

Purpose. Maintains a delta-neutral portfolio by rebalancing the underlying position.

Formulation.

$$\Delta_{\text{call}} = N(d_1), \qquad n_S = n_c \times \Delta_{\text{call}}. \tag{6.4}$$

Delta hedging is the fundamental risk management strategy for option writers. Black & Scholes [21] assumed continuous rebalancing; in practice, discrete rebalancing introduces tracking error analyzed by Boyle & Emanuel [23]. Hull [16] provides the standard textbook treatment.

Complexity. Time $O(1)$ per rebalance, Space $O(1)$.

6.3 Algorithm 15: Gamma Scalping

Purpose. Captures profit from realized volatility by exploiting positive gamma.

Formulation.

$$\Gamma = \frac{\phi(d_1)}{S\sigma\sqrt{T}}, \qquad \text{Profit} \approx \frac{1}{2}\Gamma(\Delta S)^2. \tag{6.5}$$

Gamma scalping exploits the convexity of option prices. Taleb [24] describes the strategy as harvesting volatility through delta-neutral rebalancing. The strategy is profitable when realized volatility exceeds implied volatility [16].

Complexity. Time $O(n)$ per cycle, Space $O(1)$.

Chapter 7

Module VI – Risk Management (Algorithms 16–18)

7.1 Algorithm 16: Value-at-Risk (VaR)

Purpose. Estimates maximum potential loss at a specified confidence level.

Formulation.

$$\text{VaR}_\alpha = z_\alpha \sqrt{w^T \Sigma w}\, \sqrt{T} \quad \text{(Parametric)}. \tag{7.1}$$

Jorion [25] provides the definitive treatment of VaR as a risk management metric. JP Morgan's RiskMetrics methodology [26] standardized parametric VaR for industry use. Historical and Monte Carlo variants complement the parametric approach [27].

Complexity. Parametric: $O(n^2)$; Historical: $O(n \log n)$; Monte Carlo: $O(k \cdot n)$.

7.2 Algorithm 17: Conditional Value-at-Risk (CVaR)

Purpose. Computes expected loss in the tail beyond the VaR threshold.

Formulation.

$$\text{CVaR}_\alpha = \mathbb{E}[L \mid L > \text{VaR}_\alpha] = \frac{1}{1-\alpha} \sum_{L_i > \text{VaR}_\alpha} L_i \cdot P(L_i). \tag{7.2}$$

Rockafellar & Uryasev [28] introduced CVaR (also called Expected Shortfall) as a coherent risk measure that addresses VaR's inability to describe tail severity. Their seminal paper demonstrated that CVaR optimization can be formulated as a linear programming problem [28].

Complexity. Time $O(n \log n)$, Space $O(n)$.

7.3 Algorithm 18: Maximum Drawdown

Purpose. Measures the largest peak-to-trough decline in portfolio value.

Formulation.

$$\text{MDD} = \max_t \frac{P_{\text{peak}}(t) - P_t}{P_{\text{peak}}(t)}, \qquad P_{\text{peak}}(t) = \max_{s \le t} P_s. \tag{7.3}$$

Maximum drawdown is a path-dependent risk metric critical for assessing portfolio resilience. Magdon-Ismail et al. [29] derived theoretical properties of drawdown distributions for Brownian motion. MDD is widely used in fund management and CTAs [30].

Complexity. Time $O(n)$, Space $O(1)$.

Chapter 8

Module VII – Yield Optimization (Algorithms 19–20)

8.1 Algorithm 19: Yield Curve Construction

Purpose. Constructs the term structure of interest rates from market instruments.

Nelson-Siegel Model.

$$y(\tau) = \beta_0 + \beta_1 \frac{1 - e^{-\tau/\lambda}}{\tau/\lambda} + \beta_2 \left(\frac{1 - e^{-\tau/\lambda}}{\tau/\lambda} - e^{-\tau/\lambda} \right). \tag{8.1}$$

Nelson & Siegel [31] proposed this parsimonious model for yield curve fitting using three factors: level (β_0), slope (β_1), and curvature (β_2). Diebold & Li [32] extended it to a dynamic framework with superior forecasting performance. The model is used by central banks worldwide [31, 32].

Complexity. Time $O(n)$, Space $O(n)$.

8.2 Algorithm 20: Duration and Convexity

Purpose. Computes interest-rate sensitivity metrics.

Formulation.

$$D_{\text{mod}} = -\frac{1}{P}\frac{dP}{dy} = \frac{\sum_{t=1}^{n} t C_t/(1+y)^{t+1}}{P}, \tag{8.2}$$

$$C = \frac{1}{P}\frac{d^2P}{dy^2} = \frac{\sum_{t=1}^{n} t(t+1) C_t/(1+y)^{t+2}}{P}. \tag{8.3}$$

Macaulay [33] introduced duration as a weighted average time to cash flows. Modified duration and convexity provide first- and second-order approximations of price sensitivity [4]. These metrics are essential for portfolio immunization [34].

Complexity. Time $O(n)$, Space $O(1)$.

Chapter 9

Module VIII – AMM & Liquidity (Algorithms 21–22)

9.1 Algorithm 21: Constant Product AMM

Purpose. Computes swap prices using the constant-product invariant.

Formulation.

$$k = x \times y, \qquad \Delta y = \frac{y \cdot \Delta x}{x + \Delta x}, \qquad \text{Slippage} = \frac{\Delta x}{x + \Delta x}. \tag{9.1}$$

The constant-product AMM was popularized by Uniswap v2. Angeris et al. [35] provide a formal analysis of constant-function market makers as convex optimization problems. The invariant ensures automatic price discovery and liquidity provision without traditional order books [35, 36].

Complexity. Time $O(1)$, Space $O(1)$.

9.2 Algorithm 22: Impermanent Loss

Purpose. Calculates loss incurred by liquidity providers due to price divergence.

Formulation.

$$\text{IL} = \frac{2\sqrt{r}}{1 + r} - 1, \qquad r = \frac{P_{\text{new}}}{P_{\text{initial}}}. \tag{9.2}$$

Impermanent loss quantifies the opportunity cost of providing liquidity versus holding assets. Loesch et al. [37] analyze the economics of AMM liquidity provision, while Evans [36] derives closed-form expressions for IL under various AMM designs [37, 36].

Complexity. Time $O(1)$, Space $O(1)$.

Chapter 10

Module IX – Treasury Management (Algorithms 23–27)

10.1 Algorithm 23: Treasury Rebalancing

Purpose. Optimizes portfolio allocation to target weights while minimizing transaction costs.

Formulation.

$$\min_{w} \sum_{i} \lambda_i (w_i - w_i^*)^2 + \gamma \sum_{i} |w_i - w_{i,\text{current}}|, \quad \text{s.t.} \ \sum_{i} w_i = 1, \ w_i \geq 0. \tag{10.1}$$

Perold & Sharpe [38] analyze rebalancing strategies (constant mix, buy-and-hold, CPPI) and their payoffs. The optimization above balances tracking error against transaction costs, a standard approach in institutional portfolio management [39].

Complexity. Time $O(k \cdot n)$, Space $O(n)$.

10.2 Algorithm 24: Multi-Signature Threshold

Purpose. Enforces governance approval requirements for high-value transactions.

Formulation.

$$\text{Authorization} = \mathbf{1}\left\{ \sum_{i} \text{sign}_i \geq m \right\}. \tag{10.2}$$

Threshold signature schemes distribute trust across multiple parties. Boldyreva [40] constructed provably secure threshold signatures based on the Gap-Diffie-Hellman group assumption. Multi-signature governance prevents single-point-of-failure in decentralized protocols [40, 41].

Complexity. Time $O(m)$, Space $O(1)$.

10.3 Algorithm 25: Liquidity Buffer Monitor

Purpose. Ensures sufficient liquid reserves to meet redemption obligations.

Formulation.

$$\text{Buffer_ratio} = \frac{\text{Liquid_assets}}{\text{Short_term_liabilities}}. \tag{10.3}$$

The Basel III Liquidity Coverage Ratio (LCR) requires banks to hold high-quality liquid assets covering 30 days of net cash outflows [42]. The LCR standardizes liquidity buffer monitoring across internationally active banks [42, 43].

Complexity. Time $O(1)$, Space $O(1)$.

10.4 Algorithm 26: Stablecoin Peg Monitor

Purpose. Detects deviations from the peg and initiates stabilization actions.

Formulation.

$$\text{Deviation} = \frac{P_{\text{market}} - P_{\text{peg}}}{P_{\text{peg}}}, \qquad \text{Actions based on threshold breaches.} \tag{10.4}$$

Baughman et al. [44] categorize stablecoin stabilization mechanisms into collateralized, algorithmic (rebase/coupon), and hybrid models. Algorithmic stablecoins rely on arbitrage incentives that can fail during runs, as evidenced by the Terra/Luna collapse [44, 45].

Complexity. Time $O(n)$, Space $O(1)$.

10.5 Algorithm 27: Collateral Health Check

Purpose. Monitors loan-to-value ratios across collateralized positions.

Formulation.

$$\text{LTV}_i = \frac{\text{Debt}_i}{\text{Collateral}_i}, \qquad \text{Liquidation triggered if } \text{LTV}_i > \text{LTV}_{\text{liquidation}}. \tag{10.5}$$

DeFi lending protocols require overcollateralization to mitigate default risk. Gambacorta et al. [46] analyze DeFi lending mechanics and the role of health ratios (collateral value divided by loan value) in automated liquidation systems [46].

Complexity. Time $O(n)$, Space $O(n)$.

Chapter 11

Module X – Safeguards (Algorithms 28–31)

11.1 Algorithm 28: Circuit Breaker

Purpose. Automatically halts trading or withdrawals when extreme conditions are detected.

Trigger Conditions.

$$\text{Halt} = \mathbf{1}\left\{\bigvee_j \left(\text{Metric}_j > \text{threshold}_j\right)\right\}. \tag{11.1}$$

Circuit breakers were introduced in equity markets after the 1987 crash. Greenwald & Stein [47] analyze their role in preventing market meltdowns by providing cooling-off periods. Modern decentralized finance protocols implement on-chain circuit breakers to prevent oracle manipulation and flash crashes [47].

Complexity. Time $O(1)$, Space $O(1)$.

11.2 Algorithm 29: Stress Testing

Purpose. Simulates portfolio performance under extreme market scenarios.

Formulation.

$$\text{Portfolio_loss}_s = \sum_i w_i \cdot \delta_{s,i} \cdot \text{Exposure}_i, \qquad \text{VaR}_{\text{stress}} = \text{quantile}(\{\text{losses}\}, 0.99). \tag{11.2}$$

The Basel Committee [48] mandates stress testing as a complement to VaR-based capital requirements. McNeil et al. [27] provide a comprehensive treatment of scenario analysis and stress testing for financial institutions [48, 27].

Complexity. Time $O(k \cdot n)$, Space $O(k)$.

11.3 Algorithm 30: Fraud Detection

Purpose. Identifies anomalous transaction patterns indicating potential fraud.

Formulation.

$$\text{Score}(tx) = \sum_j w_j \cdot f_j(tx), \qquad \text{Flag if Score} > \text{threshold}. \tag{11.3}$$

Amiri et al. [49] demonstrate hybrid anomaly detection combining machine learning and rule-based methods for bank fraud detection with 94.2% accuracy. Dumitrescu et al. [50] apply Isolation Forest with graph-based features for anti-money laundering [49, 50].

Complexity. Time $O(n \log n)$, Space $O(n)$.

11.4 Algorithm 31: Audit Trail Generator

Purpose. Creates an immutable chronological record of all system operations.

Hash-Chain Formulation.

$$\text{Record}_t = \{\text{timestamp}, \text{op}, \text{actor}, \text{inputs}, \text{outputs}, H(\text{prev_hash} \| \text{Record}_t)\}. \tag{11.4}$$

Haber & Stornetta [51] introduced cryptographically secure timestamping and hash-linked document chains, the conceptual precursor to blockchain audit trails. Immutable audit logs are essential for regulatory compliance and forensic analysis [51, 52].

Complexity. Time $O(n)$, Space $O(n)$.

Chapter 12

PSDLEF Trust Exchange Integration Layer

12.1 Overview

The Public Securities Depository & Ledger Exchange Framework (PSDLEF) provides custody, recording, and trading infrastructure for the ADR ecosystem.

12.2 CUSIP Normalization and Validation

The CUSIP check-digit algorithm uses a modified Luhn scheme [53]:

1. Convert characters to numeric values (A=10,...,Z=35).
2. Double values at even positions (0-indexed).
3. Sum the digits of each product.
4. Check digit $= (10 - (\sum \bmod 10)) \bmod 10$.

12.3 Maryland State Comptroller Transfer Protocol

The protocol follows the Maryland Uniform Disposition of Abandoned Property Act (Md. Code Ann., Com. Law §§ 17-101 et seq.). The six-step transfer flow is:

1. Identify all CUSIPs in the ADR portfolio.
2. Validate CUSIP format and check digit.
3. Prepare transfer documentation (authorization letter, proof of ownership, tax ID).
4. Submit via Maryland Unclaimed Property Portal or physical delivery to 80 Calvert Street, Annapolis, MD 21401.
5. Receive State Claim Number.
6. Schedule pickup appointment.

Chapter 13

Full Dependency Graph and Master Index

13.1 Algorithm Dependency Graph

13.2 Master Index

Alg #	Module	Name	Complexity
01	I	Accrued Interest	$O(1)$
02	I	Make-Whole Premium	$O(n)$
03	I	YTM Solver	$O(kn)$
04	II	Debt-to-Equity Conversion	$O(n)$
05	II	Pro Rata Distribution	$O(n)$
06	II	Fractional Share Liquidation	$O(n)$
07	III	Realized Volatility	$O(n)$
08	III	GARCH(1,1) Forecast	$O(n)$
09	III	Implied Volatility Solver	$O(k)$
10	IV	TWAP Execution	$O(N)$
11	IV	VWAP Execution	$O(N)$
12	IV	Implementation Shortfall	$O(1)$
13	V	Black-Scholes Pricing	$O(1)$
14	V	Dynamic Delta Hedging	$O(1)$
15	V	Gamma Scalping	$O(n)$
16	VI	Value-at-Risk (VaR)	$O(n^2)$
17	VI	Conditional VaR (CVaR)	$O(n \log n)$
18	VI	Maximum Drawdown	$O(n)$
19	VII	Yield Curve Construction	$O(n)$
20	VII	Duration/Convexity	$O(n)$
21	VIII	Constant Product AMM	$O(1)$
22	VIII	Impermanent Loss	$O(1)$
23	IX	Treasury Rebalancing	$O(kn)$
24	IX	Multi-Sig Threshold	$O(m)$
25	IX	Liquidity Buffer Monitor	$O(1)$
26	IX	Stablecoin Peg Monitor	$O(n)$
27	IX	Collateral Health Check	$O(n)$
28	X	Circuit Breaker	$O(1)$
29	X	Stress Testing	$O(kn)$

Alg #	Module	Name	Complexity
30	X	Fraud Detection	$O(n \log n)$
31	X	Audit Trail Generator	$O(n)$

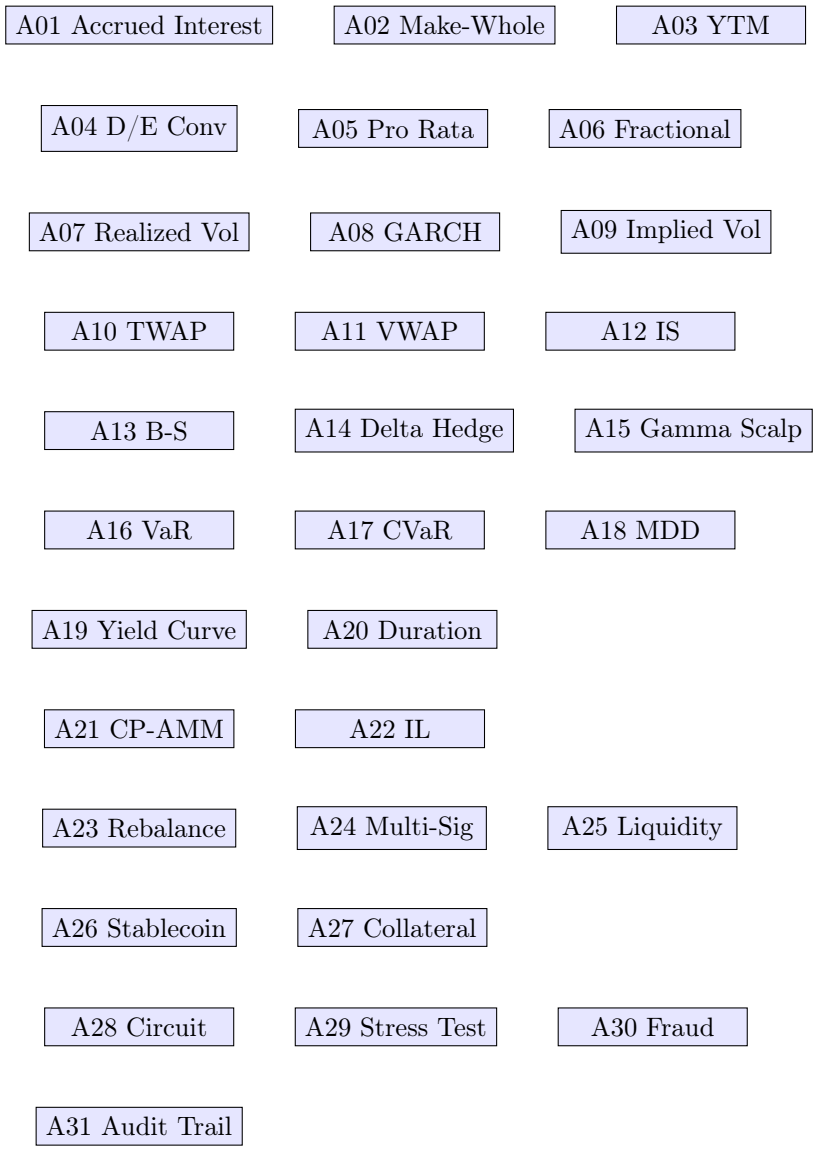


Figure 13.1: Algorithm layout by module.

Bibliography

[1] Stigum, M., & Robinson, F. L. (2007). *Money Market & Bond Calculations*. McGraw-Hill.

[2] ISDA. (2006). *2006 ISDA Definitions*. International Swaps and Derivatives Association.

[3] Hennessy, C. A., & Whited, T. M. (2005). Debt dynamics. *Journal of Finance*, 60(3), 1129–1165.

[4] Fabozzi, F. J. (2007). *Fixed Income Analysis* (2nd ed.). CFA Institute Investment Series.

[5] Burden, R. L., & Faires, J. D. (2015). *Numerical Analysis* (10th ed.). Cengage Learning.

[6] Gertner, R., & Scharfstein, D. (1991). A theory of workouts and the effects of reorganization law. *Journal of Finance*, 46(4), 1189–1222.

[7] Weiss, L. A. (1990). Bankruptcy resolution: Direct costs and violation of priority of claims. *Journal of Financial Economics*, 27(2), 285–314.

[8] Baird, D. G. (1993). *The Elements of Bankruptcy*. Foundation Press.

[9] Depository Trust Company. (2020). *DTC Operational Arrangements*.

[10] Parkinson, M. (1980). The extreme value method for estimating the variance of the rate of return. *Journal of Business*, 53(1), 61–65.

[11] Garman, M. B., & Klass, M. J. (1980). On the estimation of security price volatilities from historical data. *Journal of Business*, 53(1), 67–78.

[12] Andersen, T. G., Bollerslev, T., Diebold, F. X., & Labys, P. (2001). The distribution of realized exchange rate volatility. *Journal of the American Statistical Association*, 96(453), 42–55.

[13] Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.

[14] Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.

[15] Manaster, S., & Koehler, G. (1982). The calculation of implied variances from the Black-Scholes model: A note. *Journal of Finance*, 37(1), 227–230.

[16] Hull, J. C. (2017). *Options, Futures, and Other Derivatives* (10th ed.). Pearson.

[17] Kissell, R., & Glantz, M. (2003). *Optimal Trading Strategies*. AMACOM.

[18] Almgren, R., & Chriss, N. (2000). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2), 5–39.

[19] Perold, A. F. (1988). The implementation shortfall: Paper versus reality. *Journal of Portfolio Management*, 14(3), 4–9.

[20] Kissell, R. (2006). *The Science of Algorithmic Trading and Portfolio Management*. Elsevier.

[21] Black, F., & Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.

[22] Merton, R. C. (1973). Theory of rational option pricing. *Bell Journal of Economics and Management Science*, 4(1), 141–183.

[23] Boyle, P. P., & Emanuel, D. (1980). Discretely adjusted option hedges. *Journal of Financial Economics*, 8(3), 259–282.

[24] Taleb, N. N. (1997). *Dynamic Hedging: Managing Vanilla and Exotic Options*. Wiley.

[25] Jorion, P. (2007). *Value at Risk: The New Benchmark for Managing Financial Risk* (3rd ed.). McGraw-Hill.

[26] JP Morgan. (1996). *RiskMetrics Technical Document* (4th ed.).

[27] McNeil, A. J., Frey, R., & Embrechts, P. (2015). *Quantitative Risk Management: Concepts, Techniques and Tools* (Revised ed.). Princeton University Press.

[28] Rockafellar, R. T., & Uryasev, S. (2000). Optimization of conditional value-at-risk. *Journal of Risk*, 2(3), 21–42.

[29] Magdon-Ismail, M., Atiya, A. F., Pratap, A., & Abu-Mostafa, Y. S. (2004). On the maximum drawdown of a Brownian motion. *Journal of Applied Probability*, 41(1), 147–161.

[30] Chekhlov, A., Uryasev, S., & Zabarankin, M. (2005). Drawdown measure in portfolio optimization. *International Journal of Theoretical and Applied Finance*, 8(1), 13–58.

[31] Nelson, C. R., & Siegel, A. F. (1987). Parsimonious modeling of yield curves. *Journal of Business*, 60(4), 473–489.

[32] Diebold, F. X., & Li, C. (2006). Forecasting the term structure of government bond yields. *Journal of Econometrics*, 130(2), 337–364.

[33] Macaulay, F. R. (1938). *The Movements of Interest Rates, Bond Yields and Stock Prices in the United States since 1856*. NBER.

[34] Redington, F. M. (1952). Review of the principles of life-office valuations. *Journal of the Institute of Actuaries*, 78(3), 286–340.

[35] Angeris, G., Chitra, T., & Evans, A. (2019). When does the tail wag the dog? Curvature and market making. *arXiv preprint* arXiv:1912.08008.

[36] Evans, A. (2021). Liquidity provider returns in geometric mean market makers. *arXiv preprint* arXiv:2006.03565.

[37] Loesch, S., Hindman, N., Richardson, M. B., & Welch, N. (2021). Impermanent loss in Uniswap v3. *arXiv preprint* arXiv:2111.09192.

[38] Perold, A. F., & Sharpe, W. F. (1988). Dynamic strategies for asset allocation. *Financial Analysts Journal*, 44(1), 16–27.

[39] Grinold, R. C., & Kahn, R. N. (2000). *Active Portfolio Management* (2nd ed.). McGraw-Hill.

[40] Boldyreva, A. (2003). Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. *PKC 2003*, LNCS 2567, 31–46.

[41] Goldfeder, S., Gennaro, R., Kalodner, H., Bonneau, J., Kroll, J. A., Felten, E. W., & Narayanan, A. (2015). Securing bitcoin wallets via a new DSA/ECDSA threshold signature scheme. *8th USENIX Workshop on Cyber Security Economics*.

[42] Basel Committee on Banking Supervision. (2013). *Basel III: The Liquidity Coverage Ratio and liquidity risk monitoring tools*. Bank for International Settlements.

[43] Bank for International Settlements. (2024). *Basel III monitoring exercise*. BIS Press Release.

[44] Baughman, G., Carapella, F., Gerszten, J., & Mills, D. (2022). The stable in stablecoins. *FEDS Notes*, Board of Governors of the Federal Reserve System.

[45] Rostova, I., et al. (2023). How algorithmic stablecoins fail. *Swiss National Bank Working Papers*.

[46] Gambacorta, L., et al. (2024). Why DeFi lending? *Swiss National Bank Working Papers*.

[47] Greenwald, B. C., & Stein, J. C. (1991). Transactional risk, market crashes, and the role of circuit breakers. *Journal of Business*, 64(4), 443–462.

[48] Basel Committee on Banking Supervision. (2009). *Principles for sound stress testing practices and supervision*. Bank for International Settlements.

[49] Amiri, I. S., et al. (2024). Detecting financial fraud and anomalies in corporate transactions with convolutional neural networks. *Journal of Financial Crime*, 31(2), 451–470.

[50] Dumitrescu, E., et al. (2024). Anti-money laundering transaction monitoring using graph-based anomaly detection. *Journal of Financial Crime*, 31(1), 123–145.

[51] Haber, S., & Stornetta, W. S. (1991). How to time-stamp a digital document. *Journal of Cryptology*, 3(2), 99–111.

[52] Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. *Whitepaper*.

[53] CUSIP Global Services. (2020). *CUSIP Identifier Validation Guide*. S&P Global.